

Most Useful Commands

`git status`
`git add <file>`
`git commit -m "msg"`
`git log --oneline --graph`
`git diff`
`git diff --staged`
`git switch <branch>`
`git switch -c <branch>`
`git branch -d <branch>`
`git fetch`
`git pull`
`git push`

What's changed, staged, unstaged
Mark change for next commit
Permanently record staged changes
Readable history graph
Unstaged vs. last commit, line-by-line
What's about to be committed
Move HEAD to another branch
Create branch + switch to it
Delete branch (if merged)
Download remote changes, don't merge
Fetch + merge into current branch
Upload commits to remote

Undo Commands — Know the Difference

`git restore <file>`
`git reset --soft HEAD~1`
`git reset --hard HEAD~1`
`git revert <hash>`
`git stash`
`git stash pop`

Discard uncommitted changes
Undo commit, keep staged
Undo commit, discard all
New commit reversing an old one
Shelve uncommitted changes
Restore shelved changes

✔ local
⚠ local only
⚠ local only
✔ safe shared
✔ local
✔ local

Rule: `reset` rewrites history — unpushed commits only. `revert` adds new history — safe anytime, including on main.

Conflict Resolution Workflow

- Run the merge/rebase/cherry-pick — Git names the conflicted files
- Open file(s), read the markers:

```
<<<<<< HEAD
(what you currently have)
=====
(what's coming in)
>>>>>> branch-name
```

- Decide: keep yours / theirs / combine both
- Delete every marker line
- `git diff` — confirm zero markers remain
- `git add <file>`
- Finish: `commit` (merge) · `rebase --continue` · `cherry-pick --continue`

Merge vs. Rebase — Decision Tree

Has anyone else already pulled/pushed this branch's commits?

NO → safe to rebase
`git rebase main`
→ clean linear history, new hashes

YES → merge instead
`git merge main`
→ preserves history, 2-parent commit

Rebase your own local, unpushed work. **Never** rebase a branch other people already work from.

GitHub Workflow

- `git switch -c <branch>` — never work on main directly
- Commit with a clear message
- `git push -u origin <branch>`
- Open a Pull Request into main
- Request review from a teammate
- Address feedback, push updates
- Approved + checks pass → Merge
- Delete the branch

Branch protection basics: require PR · require ≥1 approval · require status checks · no direct pushes to main, even for admins.

Recovery Commands — the Safety Net

"I think I lost a commit"

"Found it — now what?"

"Need just that commit elsewhere"

"Detached HEAD, confused"

"Undo a commit already shared"

`git reflog`
`git branch recovered-work <hash>`
`git cherry-pick <hash>`
`git switch main` — nothing's lost
`git revert <hash>` — never reset --hard

`git reflog` keeps a record of everywhere HEAD has pointed for ~90 days by default. **Almost nothing committed in Git is ever truly gone.**